

I Built Interview Memory With Hindsight

Most AI interview coaches forget everything the moment you close the tab. I spent three weeks building one that doesn't.

The Problem No One Talks About

Every mock interview tool I've used treats each session as a clean slate. You practice answering "tell me about a system you designed under pressure" on Monday, stumble on the same question Friday, and the tool has no idea you've answered it before. No pattern recognition, no progress model, just a fresh LLM prompt generating fresh feedback into the void.

That's not coaching. That's a quiz.

When I started building CareerPilot AI — an end-to-end interview and career coaching platform — I knew the core technical challenge wasn't generating good questions. It was remembering the answers. More specifically, it was building a system that accumulates a coherent picture of a candidate across sessions, notices when they're consistently weak on behavioral framing, and adjusts accordingly. For that, I reached for Hindsight, an agent memory layer designed exactly for this kind of persistent, session-aware recall.

What CareerPilot AI Does

The platform covers the full pre-offer lifecycle: ATS resume analysis, adaptive mock interviews (text and voice), a skill gap analyzer benchmarked against real job descriptions, cover letter drafting, salary range prediction, and a career coach chat. Users set a target role — say, AI Engineer at OpenAI — and every module in the system is oriented toward closing the gap between where they are and what that role requires.

The interview simulator is the backbone. It generates role-specific questions across three categories — technical/system design, behavioral (STAR format), and general coding — calibrated to a difficulty level the user chooses. The coach evaluates each answer on structure, technical correctness, and specificity, then provides a score and model reference answer. That part is straightforward LLM prompting.

What makes it interesting is what happens with those scores over time.

The Memory Layer

Hindsight provides a hosted vector memory layer — a way to retain, recall, and reflect on information across agent sessions without building a custom retrieval pipeline from scratch. The mental model is simple: after each session, you write structured observations to Hindsight. Before each new session, you retrieve the relevant ones.

The tricky part is deciding *what* to write and *how* to structure it for meaningful retrieval later.

Here's the core pattern I settled on for session write-back:

javascript

```
async function persistSessionMemory(userId, sessionData) {
  const memoryPayload = {
    userId,
    sessionId: sessionData.id,
    role: sessionData.targetRole,
    scores: sessionData.questionScores,
    weakPatterns: sessionData.weakPatterns, // e.g. ["lacks metric outcomes", "STA
    strongSignals: sessionData.strongSignals, // e.g. ["good system decomposition"]
    timestamp: new Date().toISOString()
  };

  await hindsight.retain(memoryPayload, {
    namespace: `user:${userId}`,
    tags: ['interview', sessionData.targetRole]
  });
}
```

The `namespace` scoping is important. Without it, retrieval bleeds across users. The `tags` let me filter by role — if a user switches from "Backend Developer" prep to "AI Engineer" prep, I don't want their Django debugging sessions contaminating system design recall.

On the next session start, retrieval looks like this:

javascript

```
async function loadCandidateContext(userId, targetRole) {
  const memories = await hindsight.recall({
    namespace: `user:${userId}`,
    query: `interview performance ${targetRole}`,
    topK: 5,
    tags: ['interview', targetRole]
  });

  return memories.map(m => m.content);
}
```

The `query` isn't just a filter — it's a semantic search. If the user practiced "system design at scale" questions last week, those memories surface when today's session involves "distributed caching" even without exact string matches. That's the part that feels like real continuity rather than a glorified database lookup.

What the System Prompt Looks Like

Once I have the recalled memories, I inject them into the interview coach's context before

generating the first question:

javascript

```
async function buildCoachPrompt(userProfile, memories) {
  const memoryContext = memories.length > 0
    ? `Based on previous sessions:\n${memories.map(m =>
      `- ${m.weakPatterns.join(', ')} (${m.role}, ${m.timestamp.split('T')[0]})`
    ).join('\n')}`
    : 'No prior session history found.';

  return `You are an elite interview coach for ${userProfile.targetRole} at ${userPr

  ${memoryContext}

  Use this history to:
  1. Avoid repeating questions the candidate has already answered well.
  2. Probe harder on areas where they showed structural weakness.
  3. Reference specific prior patterns when giving feedback.

  Begin the session with a question calibrated to their current readiness level.`;
}
```

The result is a coach that opens with something like: "Last session you struggled with quantifying impact in your behavioral answers — let's focus there today." That one sentence changes the entire psychology of the practice session. The candidate knows the system has been paying attention.

The Readiness Score Problem

One of the trickiest design decisions was the Interview Readiness Score — a single percentage shown on the dashboard that's supposed to represent how ready the candidate is for their target role.

The naive version computes it from the most recent session only. That's noisy. A strong session after a bad week inflates the number; a rough session after a breakthrough week tanks it.

I use Hindsight's agent memory to maintain a rolling signal instead. After each session, I retrieve the last five entries, compute a weighted average (recent sessions count more), and update the score. The weights reflect recency without ignoring the trend:

javascript

```
function computeReadinessScore(sessions) {
  const weights = [0.35, 0.25, 0.20, 0.12, 0.08]; // index 0 = most recent
  const sorted = sessions.sort((a, b) =>
    new Date(b.timestamp) - new Date(a.timestamp)
  );

  return sorted.slice(0, 5).reduce((sum, session, i) => {
    return sum + (session.averageScore * (weights[i] || 0.05));
  }, 0);
}
```

The readiness score on the dashboard — currently showing 45% for a fresh user — stabilizes quickly once a few sessions accumulate. It becomes a number the candidate can actually trust, because it reflects weeks of signal, not a single lucky run.

The CascadeFlow Routing Layer

The platform runs across several distinct AI tasks: resume parsing, interview question generation, answer grading, coach chat, skill gap analysis, cover letter drafting, salary prediction. Not all of these need the same model.

Salary predictions and quick skill gap lookups don't require the same reasoning depth as evaluating a nuanced behavioral answer or generating a customized cover letter. I used CascadeFlow to handle model routing at runtime — cheaper, faster models for the lightweight tasks and the full context window for the ones that require it.

The CascadeFlow routing configuration lets you define rules based on task type, token budget, and latency tolerance. In practice this means a salary range estimate resolves in about 400ms using a smaller model, while an interview answer evaluation — which needs to hold the rubric, the question, the user's response, and the memory context simultaneously — gets routed to a larger context window. The admin panel shows aggregate API token cost; without routing, that number climbs fast.

Concrete Behavior: Before and After Memory

Early in testing, without memory enabled, here's what a returning user experienced:

- Session 1: Weak on quantifying outcomes, coach notes this, provides feedback.
- Session 2: Coach generates entirely fresh questions, no reference to prior gaps. User repeats the same structural mistake. Coach provides similar feedback.
- Session 3: Same thing. The user is in a loop they can't see.

With Hindsight persisting session observations:

- Session 2: Coach opens with a targeted behavioral question specifically designed to test quantification. User attempts it. Coach's feedback says: "Improvement from last session on

structure, but still no specific metric cited."

- Session 3: User finally anchors the answer with a real number. Coach's feedback acknowledges the progression explicitly.

That third session moment — where the system notices the improvement — is the part that actually changes behavior. Candidates adjust when they understand they're being tracked across a timeline, not re-evaluated from scratch each time.

Lessons I'd Take to the Next Build

Namespace your memory from day one. I retrofitted namespacing midway through development and it required replaying all existing session writes. Design your memory topology before you start persisting anything.

What you write is as important as whether you write. The first version stored raw evaluation text. Retrieval results were verbose and hard to inject cleanly into prompts. Structured, terse payloads — weak patterns as arrays, strong signals as tags — retrieve and compose much better.

Semantic recall needs a well-formed query. `(hindsight.recall({ query: "performance" })))` returns noise. `(hindsight.recall({ query: "behavioral interview STAR structure AI Engineer" })))` returns the signal you need. Treat the query as a prompt, not a keyword.

Don't let memory bloat the context window. Retrieving 20 prior sessions and dumping them into the system prompt tanks response latency and quality. `(topK: 5)` with a recency bias gives the model enough history to be contextually aware without getting confused.

Route aggressively. The difference between routing a simple task to a lighter model and sending everything to a flagship model adds up fast at scale. CascadeFlow made this configurable without touching the application logic — that's the right abstraction boundary.

CareerPilot AI is live at regal-scone-c39c6b.netlify.app. The full source is on [GitHub](#). If you're building anything where an agent needs to accumulate knowledge about a user across sessions, the combination of [Hindsight](#) for memory and [CascadeFlow](#) for intelligent routing is worth the look — the infrastructure overhead that used to eat a week of setup is now an afternoon.